



COMMUNITY GUIDE

How native Power Glove emulation works

Follow hand recognition through joystick and native game input.

2026 EDITION / IAIN BENNETT / MIT LICENSE

PowerGlove Vision offers two ways to turn the same recognized hand into game input. Most supported games use ordinary NES-style directions and buttons. Super Glove Ball can instead consume a native Power Glove packet through **Nestopia (PowerGlove)**, the separate [lr-nestopia-powerglove](#) core.

The distinction is what the game receives. It does not require a second camera system or a different hand calibration.

Follow one hand movement

1. The **PowerGlove Vision Controller (Arduino UNO Q)** captures the newest camera frame. Older waiting frames are replaced rather than queued.
2. MediaPipe Hands finds hand landmarks. Shared calibration and gesture processing turn them into position, depth, finger, and pose states.
3. The Controller sends authenticated state to RetroPie. The receiver validates it and publishes the newest usable state.
4. The chosen emulator core presents that state as the kind of controller input the game understands.
5. The game updates its world, and the display shows the result.

Each stage takes time. Smooth movement does not imply zero latency, and a successful send is not proof that a displayed frame has caught up.

Joystick-style input: directions and buttons

With FCEUmm, RetroPie exposes the **PowerGlove Vision** virtual gamepad. A profile maps recognized gestures to D-pad directions, A, B, Start, and Select. Moving sufficiently left of your saved centre can press Left; returning toward centre releases it. Activation and release thresholds help avoid repeated presses near the boundary.

This is useful for games expecting a conventional controller. Programs A-I change which gestures produce those controls. They do not teach the game to understand continuous hand coordinates. FCEUmm remains an explicit, complete joystick-style fallback for Super Glove Ball.

Native input: a position inside a packet

In the native path, the receiver also publishes a small latest-state record at </run/powerglove/native-state>. The custom core takes a coherent snapshot at the start of its input callback and converts it into the emulated Power Glove's packet. It does not accumulate a queue of past movements.

For the validated Super Glove Ball ROM, the game reads a ten-byte sample. Its fields include detection, X/Y position, depth, hand pose, a button field, and a validation terminator. Moving your hand changes a coordinate instead of only switching a direction on or off. That is why this path can offer more natural Robo-Glove positioning.

MediaPipe Hands supplies every live coordinate. Geometry is validated and the point is clamped to the saved reach before either mode sees it. **Latest coordinate** uses each newest point directly; **Bounded speed curve** uses one two-dimensional noise-aware weight to stabilize rest and follow faster movement more directly. Both use the selected frame's capture time, saved center, and per-player reach. A short missed observation may hold only X/Y for up to 120 ms while actions release; recovery starts from the first new point instead of travelling through an old or off-screen position.

What you do	Native Super Glove Ball behavior confirmed in live play
Move the hand horizontally or vertically	Continuous Robo-Glove X/Y positioning
Open the hand	Release or throw
Close the hand	Grab or catch
Point with the index finger	Fire Robo-Bullets
Make a fist and push forward	Power Punch
Use the Start gesture	Start the game

These findings include a successfully completed game. They apply to the exact ROM and implementation documented in the [native compatibility record](#). They are not a claim about every Power Glove-compatible game or ROM revision.

What stays deliberately neutral

Wrist rotation and other unmapped packet controls remain neutral. No confirmed Super Glove Ball action in the completed session required them. Bytes 7-8 stay at Nestopia's fixed \$00 initialization. Their gameplay purpose is not established; successful play at zero does not prove the ROM ignores them.

A field should only be enabled when a repeatable game behavior and a controlled test justify it. Guessing from a packet diagram can introduce unintended actions. The [packet table](#) separates confirmed meanings from the parts still under investigation.

Menu Guard suppresses ordinary D-pad and button output but does not freeze native continuous positioning. Select **Stop controller** when you want to reposition without sending controls.

Why the custom core is separate

The modified core preserves stock Nestopia and leaves FCEUmm available. RetroPie selects it for the chosen ROM rather than changing every NES game. Its launch configuration attaches the emulated Power Glove before the game's detection sequence begins.

The receiver's shared record and the ROM's packet are different formats: the record is a 64-byte host interface; the game reads the ten-byte emulated packet. Their versioning and timing should not be confused with the signed network protocol used between the two computers.

The build uses a pinned upstream revision and a maintained patch. Licensing, source provenance, and distribution details are in [Third-party components](#). Follow the [native installation instructions](#) to select the core or return that ROM to FCEUmm.

What happens when tracking or delivery fails

Lost tracking, missing calibration, the wrong profile, invalid state, or stale samples produce neutral input. The receiver and native consumer use freshness checks, including a 250-millisecond stale-state limit. Starting delivery also requires an armed Controller and an eligible game or deliberate manual context. These safeguards prevent an old movement from being held indefinitely.

They do not replace good camera placement. Recognition can still be interrupted when a hand leaves the image or becomes obscured. Keep a conventional controller available for setup and recovery.

What the responsiveness evidence means

Deterministic headless comparisons establish input handling in the emulator and game. They do not include exposure time, inference, the real network, or cabinet display latency. Live native movement is playable and substantially improved. A recent Dashboard-closed sample measured about 52 ms median MediaPipe work and about 16 new native coordinates per second, but noticeable end-to-end latency remains; the synchronized physical hand/display baseline is still pending.

The planned measurement uses one recording containing both the real hand and screen, plus separate software timings. Only after identifying the dominant stage should a tuning change be compared against the same baseline. See the [latency and jitter benchmark](#). An extra queue or more smoothing would not be a substitute for measuring the delay.